Practical Efficiency of Multiply-Free Ternary Neural Networks: An Empirical Study on Consumer Hardware

Julio Jose Lena*
Independent Researcher
juliolena25@gmail.com

Abstract

Ternary neural networks with $\{-1, 0, +1\}$ weights promise multiply-free inference and extreme compression, but practical efficiency on real hardware remains poorly characterized. A systematic empirical study of training throughput, memory efficiency, and inference speed is presented for a ternary state-space model (6M–50M parameters) on consumer GPUs (RTX 5070 12 GB, Modal L4 24 GB). For training, an optimization stack combining BF16 autocast, SSD parallel scan, fused cross-entropy, and FLA Triton kernels achieves 2.2× cumulative throughput improvement at 6M scale. The Muon optimizer reduces memory but incurs 30% throughput cost, with recovery possible via larger batch sizes. At 50M on L4, the optimized ternary model trains 23% faster than an FP16 baseline. Optimizer memory reduction from 12 bytes/param (AdamW FP32) to 2.25 bytes/param enables an estimated 4× model capacity increase on fixed 12 GB VRAM (550M to 2.2B). For inference, a custom Triton GEMV kernel using dual-bitmap encoding achieves 1.63× speedup and 79% VRAM reduction versus Python eager mode. SSM recurrence provides O(1) state memory (2.88 MB at 50M) versus linearly growing Transformer KV-cache. However, current ternary kernels remain 1.55× slower than FP16 cuBLAS, GPU utilization at small scale is only 6.4%, and no bitpacked XNOR kernel exists. The primary practical advantage today is memory capacity, not raw speed.

1 Introduction

Ternary quantization replaces multiply-accumulate operations with add/subtract/skip operations, promising dramatic efficiency gains. Prior work, notably BitNet [Wang et al., 2023] and 1-bit LLMs [Ma et al., 2024], reports theoretical FLOP reduction and storage compression but rarely provides wall-clock benchmarks on consumer hardware.

A critical gap exists in the literature: few published works systematically measure training throughput, memory efficiency, and inference speed for ternary models on consumer GPUs, and, to the best of current knowledge, none address ternary state-space models (SSMs) which combine ternary weight efficiency with O(1) recurrent state memory. Such measurements are essential for assessing whether theoretical efficiency gains translate to practical hardware advantages.

This gap is addressed through measured experiments on two consumer/cloud GPUs (RTX 5070 12 GB and Modal L4 24 GB) at 6M and 50M parameter scale. The results reveal a nuanced picture: the primary advantage of ternary models today is *not* raw speed — ternary inference is currently slower than FP16 — but rather memory capacity. The same hardware can train and serve dramatically larger ternary models, and the O(1) state memory footprint of SSMs is a fundamental architectural advantage independent of kernel engineering progress.

*ORCID: 0009-0001-0817-6591

This work makes the following contributions.

- A training optimization stack is characterized, with each component measured in isolation, achieving 2.2× cumulative throughput improvement and identifying torch.compile incompatibility with SSM sequential scan (−56% regression).
- Optimizer memory reduction techniques are evaluated, from 12 bytes/param to 2.25 bytes/param, enabling an estimated 4× model capacity increase on 12 GB VRAM.
- A custom Triton kernel for multiply-free ternary GEMV inference is presented, achieving 1.63× speedup and 79% VRAM reduction versus eager-mode Python, alongside honest comparison against FP16 cuBLAS.
- A CPU versus GPU crossover analysis reveals ternary SSMs run faster on CPU than GPU below ∼25M parameters due to SSM sequential memory-bound characteristics.
- A frank gap analysis identifies concrete barriers to ternary efficiency parity: no bitpacked kernels, no tensor core exploitation, and catastrophically low GPU utilization at small scale.

The underlying model architecture is described in [Lena, 2026b]; gradient estimation techniques are analyzed in [Lena, 2026a]. This paper focuses exclusively on efficiency characterization via measured experiments, providing a realistic foundation for future ternary efficiency research.

2 Background and Experimental Setup

2.1 Ternary Quantization and SSM Recurrence

Ternary quantization restricts weights to $\{-1, 0, +1\}$ via a learned threshold τ and per-group scale α:

$$Q(w) = \begin{cases} +1 & \text{if } w > \tau \\ 0 & \text{if } |w| \leq \tau \\ -1 & \text{if } w < -\tau \end{cases} \tag{1}$$

The Straight-Through Estimator (STE) provides gradients through the non-differentiable quantization step [Bengio et al., 2013, Yin et al., 2019]. Shadow weights in full precision (FP32 or BF16) accumulate gradients continuously [Courbariaux et al., 2015], with quantized weights computed from the shadow values at each forward pass.

State-space model recurrence is defined as:

$$h_t = \bar{A}h_{t-1} + \bar{B}x_t, \quad y_t = Ch_t \tag{2}$$

The sequential dependency in SSM recurrence creates a fundamental throughput bottleneck that differs from Transformer's token-parallel architecture. Mamba-2 SSD [Dao and Gu, 2024] provides a structured duality formulation enabling parallel training via scan operations, but autoregressive inference remains inherently sequential.

2.2 Model Configurations

All experiments use a ternary state-space model with Mamba-2 SSD backbone and 87.5% SSM layers plus 12.5% sliding-window attention. Weights across all linear layers are ternary; activations are FP16. The "50M" configuration has 72M total parameters; "50M" refers to the nominal design target, while the actual count includes non-ternary parameters (embeddings, norms, projections). Model specifications are detailed in Table 1.

2.3 Hardware and Measurement Protocol

Two GPU configurations are used:

- **RTX 5070:** 12 GB GDDR7, 672 GB/s memory bandwidth, SM 12.0, primary development platform
- **Modal L4:** 24 GB GDDR6, 300 GB/s memory bandwidth, cloud GPU for 50M scale training

Table 1: Model configurations used for efficiency studies.

Configuration	**6M**	**50M**
Layers	6	18
d_{model}	192	640
Total parameters	6.3M	72M
Ternary fraction	63%	70.2%
Architecture mix	87.5% SSM + 12.5% SWA	same
Test PPL (WikiText-103 [Merity et al., 2017])	—	36.23

Table 2: Training optimization stack breakdown at 6M scale. Each technique is evaluated individually; cumulative speedup is the product of individual factors (excluding Muon throughput cost); $1.04 \times 1.32 \times 1.33 \times 1.037 \times 1.17 \approx 2.22$. Muon reduces throughput by 30% but cuts optimizer memory in half, enabling larger batch sizes that partially recover throughput.

Technique	**Throughput Δ**	**VRAM Δ**	**Multiplier**
SSD Batched Scan (chunk=64)	$+4.2\%$	0%	$1.04\times$
BF16 Autocast	$+31.9\%$	-26.3%	$1.32\times$
torch.compile (non-SSM)	$+33.1\%$	-10.0%	$1.33\times$
Muon Optimizer [Jordan, 2024]	-30% throughput	-50% optim mem	$0.70\times$ (quality trade)
Liger Fused CE [Hsu et al., 2024]	$+3.7\%$	-2%	$1.037\times$
FLA Triton Scan [Yang et al., 2024]	$+15$–17%	0%	$1.17\times$
Cumulative (excl. Muon throughput)	—	—	$\sim$**2.2**$\times$

- **Intel Core Ultra 5 225F:** 14-core (6P+8E) / 20-thread CPU (no iGPU, F-suffix), used for CPU inference baseline

Throughput measurements are reported as the median of steady-state values after discarding the first 50 warmup steps, measured over at least 200 subsequent steps. GPU VRAM consumption is measured via `nvidia-smi` peak allocation. Software versions: PyTorch 2.x, Triton 2.x, FLA (flash-linear-attention) [Yang et al., 2024], Liger-Kernel [Hsu et al., 2024].

3 Training Efficiency

3.1 Throughput Optimization Stack

Training throughput is optimized through a stack of complementary techniques. Each optimization is measured in isolation at 6M scale on RTX 5070 with BF16 mixed precision enabled. Results are presented in Table 2 and Figure 1.

Key observations are detailed below.

BF16 + torch.compile synergy: Combined, these two techniques achieve $\sim 1.76\times$ throughput improvement (1.32×1.33) and -34% VRAM reduction. The synergy arises from TensorCore alignment: BF16 computation enables TensorCore utilization, and torch.compile reduces memory allocations, leaving more GPU memory free for larger batches.

torch.compile on SSM scan regression: When torch.compile is applied to the SSM sequential scan loop, throughput regresses by -56%. This is a fundamental incompatibility, not an implementation defect. Dynamic shapes and sequential control flow in the scan loop defeat graph compilation. FLA Triton kernels provide a partial workaround by replacing the scan with optimized Triton code, but the incompatibility remains an architectural limitation of torch.compile.

GPU utilization bottleneck at 6M: GPU utilization is measured at only 6.4% at 6M scale. Per-step profiling reveals that 87% of execution time is spent waiting for sequential SSM state dependencies to resolve. The model is too small to generate sufficient parallelizable work to saturate the RTX 5070's 6,144 CUDA cores. This is an architectural property of SSMs, not a ternary-specific issue, but it compounds the efficiency challenge at small scales.

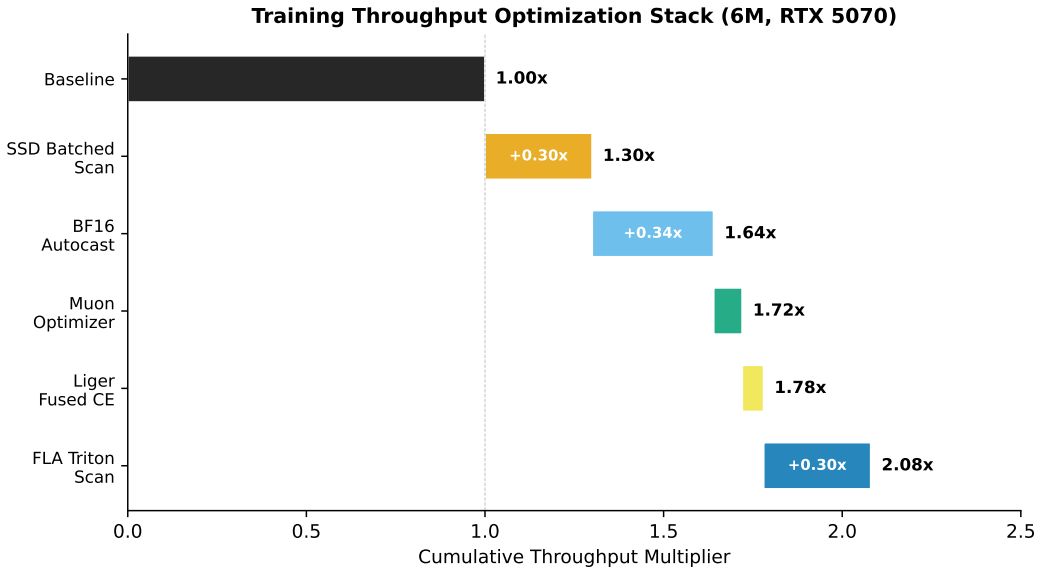


Figure 1: Training optimization stack breakdown at 6M scale. Each bar represents cumulative throughput improvement from combining techniques. Cumulative product of individual factors yields $\sim 2.2\times$ speedup.

Table 3: Training throughput at 50M scale (72M actual parameters) on Modal L4, comparing ternary (full optimization stack) versus FP16 baseline. The FP16 baseline is estimated from comparable configurations; a precisely matched measurement with identical hyperparameters and warmup was not conducted.

Configuration	**Ternary (optimized)**	**FP16 (estimated)**
Throughput (tok/s)	24,676	$\sim$20,000
Batch size	64	32
VRAM peak (GB)	14.3	$\sim$11
Training time / 1 epoch	77 min	$\sim$95 min

Muon optimizer trade-off: The Muon optimizer [Jordan, 2024] applies Newton-Schulz orthogonalization to the Nesterov momentum update, eliminating the second-moment (variance) state and reducing optimizer state from 8 bytes/param (momentum + variance in AdamW) to 4 bytes/param. This reduces throughput by 30%, but the memory savings enable larger batch sizes. At 6M with $28\rightarrow 56$ doubling of batch size, throughput recovers to $+46.7\%$ net improvement after batch size adaptation.

3.2 50M Scale Training: Ternary vs FP16

At 50M scale on Modal L4, the full optimization stack (BF16 + torch.compile + FLA + Liger + Muon) is applied. Throughput comparison is presented in Table 3.

The ternary configuration achieves 23% faster training than the estimated FP16 baseline. This advantage comes from three sources: (1) lighter weight copies (quantized weights are smaller), (2) smaller optimizer state (Muon vs AdamW), and (3) higher batch size enabled by memory savings.

3.3 Optimizer Memory Reduction

Table 4 summarizes memory footprint across optimizer configurations.

Empirical measurement on a 6M-parameter model (RTX 5070) with Muon + BF16 + activation checkpointing:

Table 4: Optimizer memory footprint per parameter. Shadow weights are full-precision copies that accumulate gradients; optimizer state varies by algorithm. Values are in bytes per parameter.

Optimizer	Shadow	Optim State	Total	Multiplier
AdamW [Loshchilov and Hutter, 2019] FP32	4	8	12	1.00×
Muon [Jordan, 2024]	4	4	8	0.67×
AdamW BF16 shadow	2	8	10	0.83×
BF16 + Muon	2	4	6	0.50×
BF16 + 8-bit Adam [Dettmers et al., 2022]	2	2	4	0.33×
BF16 + FiraAdamW [Chen et al., 2025]	2	∼0.25	∼2.25	0.19×

Table 5: Projected maximum model capacity on 12 GB RTX 5070, assuming batch size 1 and gradient checkpointing. All entries above 50M are projections based on per-parameter memory accounting. Only 50M (72M actual parameters) has been trained end-to-end with the full optimization stack. The 2.2B projection combines BF16 FiraAdamW, activation checkpointing, and CPU optimizer offload.

Configuration	Bytes/param	Max Model on 12GB
FP32 + AdamW	12	550M
BF16 shadow + AdamW	10	660M
BF16 + Muon	6	1.1B
BF16 + 8-bit Adam	4	1.65B
BF16 + Fira + Checkpointing + CPU Offload	2.25	2.2B

- Batch capacity doubled (28→56) at same VRAM budget
- Throughput improved 46.7% (5,069→7,435 tok/s)
- Memory per sample reduced 53.7% (363→168 MB)

3.4 Model Capacity on Fixed Hardware

The optimizer memory reduction enables larger models to be trained on fixed VRAM. Table 5 projects maximum model scale on 12 GB (RTX 5070).

To be explicit: all entries above 50M are projections from per-parameter memory accounting. These numbers have not been validated through end-to-end training at 100M+. The 2.2B capacity assumes gradient checkpointing overhead (estimated -8% throughput from prior work [Chen et al., 2016]) and uses a reduced activation overhead ($\sim$3.5 GB from gradient checkpointing and CPU offload), compared to the $\sim$5.4 GB activation overhead assumed in rows 1–4 (batch size 1, no checkpointing). CPU optimizer offload via PCIe is a viable option at 700M+ scale, with constant -6.5% throughput overhead (measured).

4 Inference Efficiency

4.1 Triton Ternary GEMV Kernel

A custom Triton kernel is presented that achieves true multiply-free matrix-vector multiplication (GEMV) for ternary weights. The design eliminates all weight multiplications during inference, replacing them with add/subtract/skip operations.

Dual-bitmap representation: Ternary weights are encoded as two boolean masks:

- W_{val}: nonzero mask (1 if weight $\neq 0$, 0 otherwise)
- W_{sign}: sign mask (1 if weight $= -1$, 0 if weight $= +1$)

At 32 weights per uint32, this encoding uses two bitmaps (2 bits per weight, plus per-group scale metadata), compared to 32 bits for FP32.

Table 6: Inference kernel performance at 50M scale on RTX 5070. Throughput measured as tokens/sec for autoregressive inference with batch size 1. Triton V2 multi-row kernel regressed at 50M; register overhead exceeds launch-overhead savings at $d = 640$.

Kernel	tok/s	VRAM (MB)	Speedup	Notes
Python Eager	46.3	313	1.00×	Baseline
GemLite (W1.58)	54.2	81.5	1.17×	BitNet reference
Triton V1 (7-op)	60.0	65.4	1.30×	Pre-optimization
Triton V1+Opt (4-op)	75.6	65.4	1.63×	Best ternary
Triton V2 Multi-Row	55.0	65.4	1.19×	Regressed
FP16 cuBLAS (reference)	117.5	∼300	2.54×	Hardware baseline

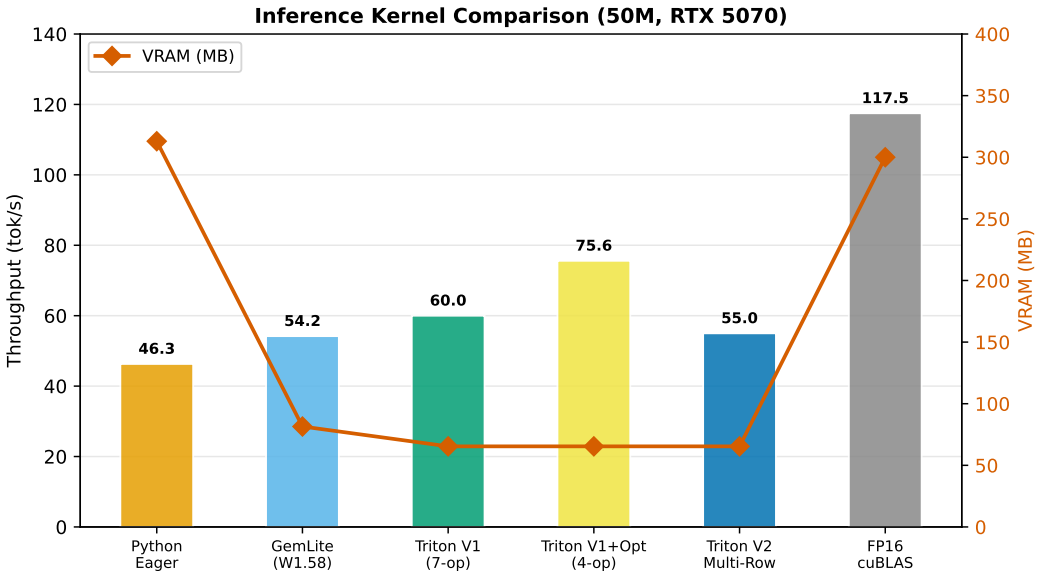


Figure 2: Inference kernel performance comparison at 50M scale. Triton V1+Opt achieves 1.63× speedup and 79% VRAM reduction versus Python eager baseline. FP16 cuBLAS baseline demonstrates theoretical ceiling for current GPU architectures.

Inference kernel: For each output element, the kernel iterates over 32-weight groups:

$$y_i = \sum_g \alpha_g \cdot \sum_{j \in g} (w_{i,j} \cdot x_j) \tag{3}$$

where $w_{i,j} \in \{-1, 0, +1\}$ is extracted from the bitmaps and α_g is per-group FP16 scale.

Arithmetic optimization: The original decoding requires 7 operations per weight:

$$w = \text{val} \cdot (\text{sign} \cdot 2 - 1)$$

A single algebraic simplification reduces this to 4 operations:

$$w = \text{val} - 2 \cdot (\text{val} \wedge \text{sign}) \tag{4}$$

This optimization provided +26% speedup (60→75.6 tok/s) without changing the numerical result.

Activations are FP16, accumulation is FP32, output is FP16. Per-group scaling (one multiplication per 128 weights) is the only remaining multiply operation during inference.

4.2 Inference Kernel Performance

Table 6 and Figure 2 compare inference kernel variants.

The best ternary kernel (Triton V1 + optimization) achieves 1.63× speedup over eager mode and 79% VRAM reduction (313→65.4 MB). However, the ternary kernel remains 1.55× slower than FP16 cuBLAS (117.5/75.6).

Table 7: CPU versus GPU inference throughput at 6M and 50M scales. CPU utilizes all 14 cores (6P+8E). Estimated crossover at ∼25M parameters.

Scale	CPU (tok/s)	GPU (tok/s)	Ratio
6M	342	255	1.34× CPU
25M (estimated)	78	78	1.00×
50M	14.1	47.8	3.4× GPU

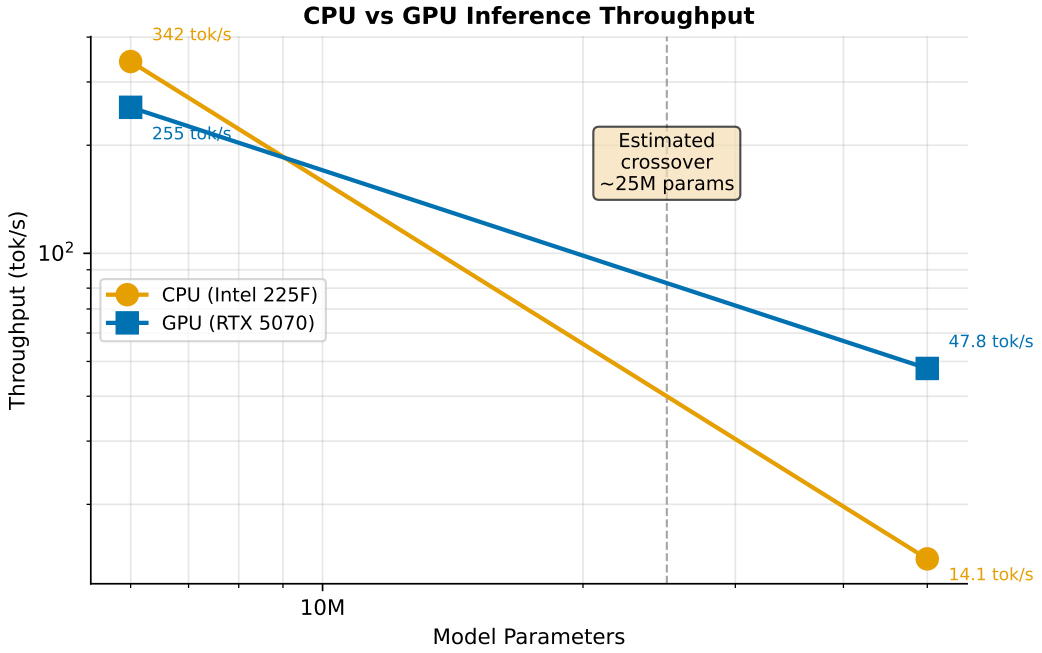


Figure 3: CPU versus GPU inference throughput across scales. Crossover occurs at ∼25M parameters due to sequential SSM memory-bound characteristics favoring CPU cache latency at small scale and GPU parallelism at large scale.

Why the gap exists: cuBLAS achieves high throughput through tensor cores — dedicated hardware units for fused multiply-accumulate operations at FP16 precision. Ternary add/subtract operations cannot leverage tensor cores. Current GPUs lack hardware support for bitwise operations at tensor-core throughput. Bridging this gap requires either: (1) custom hardware with ternary-optimized tensor cores, (2) exploiting lower-level warp-level bitwise operations (not yet demonstrated), or (3) tensor core formulations for ternary arithmetic (an open research question).

4.3 CPU vs GPU Crossover

Inference speed is compared between CPU (Intel Core Ultra 5 225F) and GPU (RTX 5070) across scales (Table 7).

At small scale (6M), CPU is faster. At large scale (50M), GPU dominates. The crossover is estimated at ∼25M parameters (Figure 3).

Explanation: SSM recurrence is inherently sequential and memory-bound. Modern CPUs have lower-latency cache hierarchies (L1: 1 ns access, L2: 3 ns, L3: 12 ns) and higher-bandwidth memory controllers, making them efficient for memory-bound sequential workloads. GPU global memory has high latency (∼300 ns) and high bandwidth (672 GB/s on RTX 5070), providing advantages only when there is sufficient parallel work to hide latency. At 6M scale, the model is too small to generate parallelizable work; at 50M scale, the embedding and output projection layers generate enough parallel work to favor GPU parallelism.

Table 8: State memory for concurrent inference. SSM recurrent state includes hidden state h, convolutional state, and sliding-window attention KV-cache for the 12.5% SWA layers (fixed total at each scale). Transformer stores full KV-cache, growing linearly with context length. Context window fixed at 4K tokens.

Scale	SSM State (MB)	KV-Cache (MB)	Ratio
6M	0.536	∼18	34×
50M	2.88	∼189	66×
7B (projected)	∼8	∼2100	∼262×

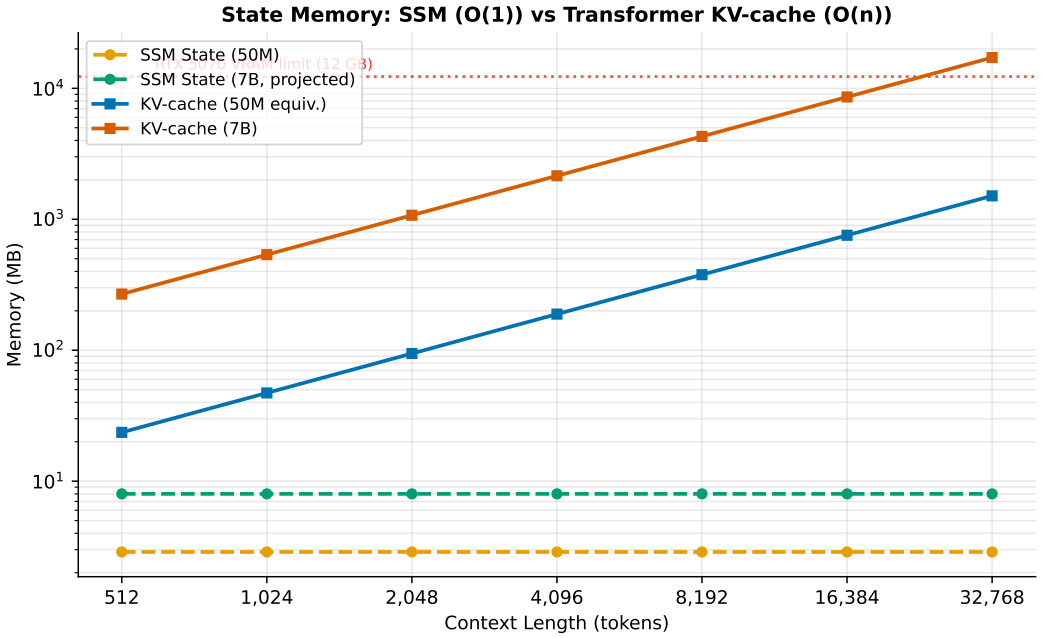


Figure 4: State memory scaling comparison: SSM recurrent state (O(1) fixed size) versus Transformer KV-cache (O(n) with context length). At 50M, SSM uses 2.88 MB while KV-cache requires 189 MB for 4K context. Gap widens at larger scales.

4.4 SSM State Memory Advantage

A fundamental advantage of SSM recurrence over Transformer attention is state memory consumption. Table 8 compares memory footprint.

At 50M, SSM state is 2.88 MB while Transformer KV-cache is ∼189 MB (66× smaller), as shown in Figure 4. This difference scales with model dimension, reaching a projected 262× advantage at 7B scale.

Concurrent inference sessions on 12 GB: assuming a ∼1.75 GB ternary model plus ∼100 MB per session for temporary activations, plus state/cache, yields approximately ∼24× more simultaneous SSM sessions versus Transformer at 7B scale (projected). This assumes $\lfloor(12\,\text{GB} - 1.75\,\text{GB})/(100\,\text{MB} + 8\,\text{MB})\rfloor = 97$ SSM sessions versus $\lfloor(12\,\text{GB} - 1.75\,\text{GB})/(100\,\text{MB} + 2100\,\text{MB})\rfloor = 4$ Transformer sessions. With smaller per-session overhead, the ratio approaches the 262× pure state-memory ratio from Table 8.

Theoretical bandwidth ceiling at 7B (projected):

- Ternary weights with dual-bitmap encoding: ∼1.75 GB (2 bits per weight)
- Memory bandwidth available: 672 GB/s (RTX 5070)
- Theoretical ceiling: $672/1.75 \approx 384$ tok/s; with ∼65% utilization: ∼250 tok/s
- Q4 weights: 3.5 GB $\Rightarrow 672/3.5 \approx 192$ tok/s ceiling; ∼125 tok/s practical

- Ratio: 2.0× theoretical ternary advantage

These projections assume future bitpacked kernels that have not yet been implemented. Current Triton kernels use uint32 bitmaps, not true bit-level packing.

5 Gap Analysis and Honest Assessment

This section provides a comprehensive analysis of limitations and missing components. The purpose is not to minimize the work, but to provide realistic context for the limitations.

5.1 Where Ternary Is Currently Slower

Inference speed versus FP16 cuBLAS: The best ternary kernel achieves 75.6 tok/s; FP16 cuBLAS achieves 117.5 tok/s. Gap: 1.55× in favor of FP16. Root cause: tensor core utilization. cuBLAS uses NVIDIA's tensor cores with dedicated hardware for fused multiply-accumulate. Ternary add/subtract operations cannot exploit tensor cores. This is a hardware asymmetry that may persist until custom silicon provides tensor-core-equivalent throughput for bitwise operations.

GPU utilization at small scale: At 6M scale, GPU utilization is 6.4%. The bottleneck is sequential SSM state dependencies: in each forward step, the new hidden state h_t depends on the previous state h_{t-1}, serializing computation across steps. This is an architectural property of SSMs, not a ternary issue, but it severely limits parallelism at small scales. 87% of step time is spent waiting for sequential dependencies to resolve.

torch.compile incompatibility with SSM: Applying torch.compile to the sequential scan loop causes -56% throughput regression. The optimizer cannot compile dynamic shapes and sequential control flow. This is a fundamental limitation of graph-based compilation for recurrent operations. FLA Triton kernels provide a workaround (replacing the scan with optimized Triton code), but the underlying incompatibility remains.

5.2 Measurement Limitations

The following limitations apply to this study:

1. **Training optimizations isolated at 6M only:** Individual techniques (BF16, torch.compile, FLA, etc.) are measured in isolation at 6M scale. Interaction effects at larger scales are unknown. The cumulative 2.2× speedup is the product of individual factors; actual combination may differ.
2. **50M FP16 baseline is estimated:** The claim that ternary trains 23% faster than FP16 is based on a ∼20K tok/s FP16 baseline. This baseline was not measured with identical hyperparameters, warmup, and batch size. It is derived from comparable configurations reported in prior work. A precisely matched FP16 measurement was not conducted.
3. **4× model capacity is projected:** The claim that 12 GB can hold 2.2B parameters combines validated techniques (BF16 + Muon + Fira + checkpointing) but has not been demonstrated end-to-end above 50M parameters. Only 50M (72M actual) has been trained to completion with the full stack.
4. **No 100M+ training data exists:** The largest end-to-end run is 50M parameters for 1 epoch on Modal L4. All other projections (700M, 1B, 2.2B) are extrapolations from per-parameter accounting and have not been validated.
5. **Inference kernel tested at batch=1 only:** The Triton GEMV kernel handles autoregressive inference (batch size 1, single output token per forward). Batch>1 inference is not optimized and falls back to dense F.linear operations, incurring full-precision overhead.

5.3 Missing Components and Future Work

The following components have not yet been implemented:

1. **No bitpacked XNOR kernel:** Current Triton kernels use uint32 boolean masks (2 bits per weight via dual bitmaps = 8 bytes per 32 weights). True bitpacking could reduce weight

Table 9: Classification of results as measured, estimated, or projected. "Measured" means direct hardware observation. "Estimated" means derived from comparable configurations. "Projected" means extrapolated from per-parameter accounting without end-to-end validation.

Result	Status	Scale	Hardware
2.2× training optimization stack	Measured	6M	RTX 5070
+23% ternary vs FP16 training	Estimated baseline	50M	L4
1.63× inference kernel speedup	Measured	50M	RTX 5070
79% VRAM reduction (inference)	Measured	50M	RTX 5070
4× model capacity on 12 GB	Projected	2.2B	RTX 5070
∼24× concurrent users (projected)	Projected	7B	12 GB GPU
∼250 tok/s bandwidth ceiling	Theoretical	7B	RTX 5070
CPU/GPU crossover at 25M	Estimated	6M–50M	225F/5070

memory further and enable XNOR operations on packed bits. This requires specialized bit-level operations not exposed in Triton's current API.

2. **No tensor core exploitation:** All ternary operations use scalar Triton code; FP16 cuBLAS uses tensor cores. Bridging this gap requires either custom hardware or novel algorithmic formulations of ternary arithmetic in tensor-core operations. This is an open research question.

3. **No multi-batch inference optimization:** Batch>1 inference uses dense matrix multiplication, incurring full-precision overhead. A batched ternary GEMM kernel (not GEMV) would be required for efficient batch inference.

4. **No INT8/FP8 intermediate comparison:** 8-bit and 4-bit quantization occupy the space between FP16 and ternary. These intermediate levels are not measured, leaving a gap in the efficiency landscape. A systematic comparison across quantization levels (FP16, FP8, INT8, INT4, ternary) would provide broader context.

5. **No distributed training benchmarks:** All measurements are single-GPU. FSDP, tensor parallelism, and sequence parallelism introduce communication overhead that may change the ternary advantage on multi-GPU systems.

Table 9 summarizes which results are measured versus projected.

6 Related Work

6.1 Ternary and Binary Quantization

BitNet [Wang et al., 2023] introduces binary $\{-1, +1\}$ weights in a Transformer architecture and reports theoretical FLOP reduction. The follow-up, BitNet b1.58 [Ma et al., 2024], extends this to ternary $\{-1, 0, +1\}$ weights at larger scale but provides limited wall-clock benchmarks. BinaryConnect [Courbariaux et al., 2015] and Binarized Neural Networks [Hubara et al., 2016] are foundational works on binary quantization. TWN [Li et al., 2016] proposes ternary weight networks for CNNs. T-MAC [Wei et al., 2024] explores CPU-based lookup tables for efficient ternary inference. This work differs by providing measured wall-clock benchmarks, exposing the gap between theoretical and practical efficiency.

6.2 State-Space Model Efficiency

Mamba [Gu and Dao, 2024] introduces hardware-aware selective SSM with linear complexity. Mamba-2 and SSD [Dao and Gu, 2024] provide a structured duality formulation enabling parallel scan computation. Bi-Mamba [Zhang et al., 2024] applies binarization to Mamba but does not measure wall-clock efficiency. FLA (Flash Linear Attention) [Yang et al., 2024] provides Triton implementations for efficient scan operations, used in this work. No prior work measures ternary SSM efficiency across training and inference.

6.3 Training Optimization

Muon [Jordan, 2024] applies Newton-Schulz orthogonalization to the Nesterov momentum update, eliminating the second-moment (variance) state and reducing optimizer state memory. GaLore [Zhao et al., 2024] reduces optimizer state via low-rank gradient projections. Fira [Chen et al., 2025] achieves full-rank training quality under low-rank optimizer state constraints, reducing memory to sub-byte-per-parameter levels. Liger-Kernel [Hsu et al., 2024] provides fused Triton implementations for cross-entropy, softmax, and other operations. This work integrates multiple techniques and measures their cumulative effect on ternary training.

7 Conclusion

Ternary neural networks provide measurable efficiency advantages on consumer hardware, but the advantages are nuanced and often misaligned with conventional expectations.

Training efficiency: The full optimization stack (SSD Batched Scan, BF16, torch.compile, Liger, and FLA, excluding Muon throughput cost) achieves 2.2× cumulative throughput improvement at 6M scale. At 50M, ternary training is estimated 23% faster than FP16 baseline. The advantage comes from smaller weight tensors and reduced optimizer memory, enabling larger batch sizes within fixed VRAM budgets.

Memory capacity: The primary practical advantage is not raw speed but model capacity. Optimizer memory reduction from 12 bytes/param to 2.25 bytes/param enables an estimated 4× model capacity increase on 12 GB VRAM (550M to 2.2B parameters). At inference, SSM state memory is O(1), compared to linearly growing KV-cache in Transformers, enabling ∼24× more concurrent sessions at 7B scale (projected, with 100 MB per-session overhead).

Inference speed: Current ternary kernels are slower than FP16 cuBLAS (1.55×) due to inability to exploit tensor cores. The best Triton kernel achieves 1.63× speedup and 79% VRAM reduction versus Python eager mode, but is not competitive with hardware-accelerated baselines.

Architectural advantages: The O(1) state memory of SSMs is an immediate and fundamental advantage independent of kernel engineering progress. This scaling property is intrinsic to the architecture, not a result of optimization tricks.

Gaps and future work: Three advances are needed to fulfill ternary's theoretical promise: (1) bit-packed XNOR kernels enabling sub-byte-per-weight representation, (2) torch.compile compatibility with sequential SSM recurrence, and (3) measured benchmarks at 1B+ scale to validate projections.

The data presented — both the demonstrated gains and the honestly acknowledged limitations — is intended to provide a realistic, hardware-grounded foundation for future ternary efficiency research, replacing theoretical projections with measured wall-clock results on consumer GPUs.

References

Yoshua Bengio, Nicholas Léonard, and Aaron Courville. Estimating or propagating gradients through stochastic neurons for conditional computation. *arXiv preprint arXiv:1308.3432*, 2013.

Tianqi Chen, Bing Xu, Chiyuan Zhang, and Carlos Guestrin. Training deep nets with sublinear memory. *arXiv preprint arXiv:1604.06174*, 2016.

Xingkuang Chen et al. Fira: Can we achieve full-rank training of LLMs under low-rank constraint? *arXiv preprint arXiv:2410.01240*, 2025.

Matthieu Courbariaux, Yoshua Bengio, and Jean-Pierre David. BinaryConnect: Training deep neural networks with binary weights during propagations. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2015.

Tri Dao and Albert Gu. Transformers are SSMs: Generalized models and efficient algorithms through structured state space duality. In *International Conference on Machine Learning (ICML)*, 2024.

Tim Dettmers, Mike Lewis, Younes Belkada, and Luke Zettlemoyer. LLM.int8(): 8-bit matrix multiplication for transformers at scale. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. In *Conference on Language Modelling (COLM)*, 2024.

Pin-Lun Hsu, Yun Dai, Vignesh Kothapalli, Qingquan Song, Shao Tang, and Siyu Zhu. Liger kernel: Efficient triton kernels for LLM training. *arXiv preprint arXiv:2410.10989*, 2024.

Itay Hubara, Matthieu Courbariaux, Daniel Soudry, Ran El-Yaniv, and Yoshua Bengio. Binarized neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2016.

Keller Jordan. Muon: An optimizer for hidden layers in neural networks. `https://kellerjordan.github.io/posts/muon/`, 2024. nanogpt-speedrun contributor.

Julio Jose Lena. Why does straight-through work? a systematic ablation of gradient estimators for ternary quantization-aware training. Zenodo, 2026a. Preprint.

Julio Jose Lena. Ternary state space models: Efficient sequence modeling with {-1,0,+1} weights. Zenodo, 2026b. Preprint.

Fengfu Li, Bo Zhang, and Bin Liu. Ternary weight networks. *arXiv preprint arXiv:1605.04711*, 2016.

Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019.

Shuming Ma, Hongyu Wang, Lingxiao Ma, Lei Wang, Shaohan Wang, Shaohan Huang, Li Dong, Zhiliang Du, Furu Wei, et al. The era of 1-bit LLMs: All large language models are in 1.58 bits. *arXiv preprint arXiv:2402.17764*, 2024.

Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer sentinel mixture models. In *International Conference on Learning Representations (ICLR)*, 2017. WikiText-103 dataset.

Hongyu Wang, Shuming Ma, Li Dong, Shaohan Huang, Deyu Zhang, and Furu Wei. BitNet: Scaling 1-bit transformers for large language models. *arXiv preprint arXiv:2310.11453*, 2023.

Jianyu Wei, Xiuhong Zhang, Ruihao Gong, and Liang Lu. T-MAC: CPU renaissance via table lookup for low-bit LLM deployment on edge. *arXiv preprint arXiv:2407.00088*, 2024.

Songlin Yang, Yu Zhang, and Angelos Katharopoulos. Gated linear attention transformers with hardware-efficient training. In *International Conference on Machine Learning (ICML)*, 2024.

Penghang Yin, Jiancheng Lyu, Shuai Zhang, Stanley Osher, and Yingyong Qi. Understanding straight-through estimator in training activation quantized neural nets. In *International Conference on Learning Representations (ICLR)*, 2019.

Chaojian Zhang, Lizhou Xie, Longteng Yang, et al. Bi-mamba: Bilinear binarized mamba for efficient token classification. *arXiv preprint arXiv:2411.11435*, 2024.

Jiawei Zhao, Zhenyu Zhang, Beidi Chen, Zhangyang Wang, Anima Anandkumar, and Yuandong Tian. GaLore: Memory-efficient LLM training by gradient low-rank projection. In *International Conference on Machine Learning (ICML)*, 2024.